

I. Fundamental Principles: Locality and Hierarchy

1. *Goal of Memory Hierarchy:* The primary goal is to create the *illusion of unlimited amounts of fast memory* [1].
2. *Principle of Locality:* This principle underlies the operation of memory hierarchies, stating that programs access only a relatively small portion of their address space at any given instant [2].
 - * *Temporal Locality (Locality in Time):* If an item is referenced, it will tend to be referenced again soon (e.g., instructions and data within loops) [2, 3]. Hierarchies exploit this by keeping recently accessed data closer to the processor [4].
 - * *Spatial Locality (Locality in Space):* If an item is referenced, items whose addresses are close by will tend to be referenced soon (e.g., sequential instruction execution or accessing array elements) [3, 5]. Hierarchies exploit this by using *blocks* that contain multiple contiguous words [4, 5].
3. *Memory Hierarchy Structure:* A hierarchy consists of multiple levels of memory with differing speeds and sizes [6]. The faster memories are more expensive per bit and thus smaller, positioned closer to the processor [4, 6, 7]. Data closer to the processor is generally a subset of the data stored at the next lower level [8].

II. Core Metrics and Terminology

1. *Block (or Line):* The minimum unit of information that can be present or absent in a level of the memory hierarchy [9, 10]. Data is typically copied between adjacent levels one entire block at a time [11].
2. *Hit vs. Miss:*
 - * *Hit:* The requested data is found in the upper level (closer to the processor) [9].
 - * *Miss:* The requested data is not found in the upper level, requiring access to the lower level [9].
3. *Performance Metrics:*
 - * *Hit Rate (or Ratio):* The fraction of memory accesses found in the upper level [9, 10].
 - * *Miss Rate:* The fraction of memory accesses not found in the upper level (Miss Rate = $1 - \text{Hit Rate}$) [9, 10].
4. *Timing Metrics:*
 - * *Hit Time:* The time required to access the upper level and determine if it is a hit or a miss [10, 12].
 - * *Miss Penalty:* The time required to fetch the block from the lower level, transmit it, insert it into the upper level, and deliver it to the processor [10, 12]. The miss penalty is typically much larger than the hit time [12].

III. Memory Technologies and Hierarchy Levels

1. *Cache Levels (L1/L2):* Implemented using *SRAM* (Static Random Access Memory) [13, 14].

- * *SRAM Characteristics:* Faster and more expensive than DRAM [13, 14]. Uses 6 to 8 transistors per bit and does not need periodic refresh [15, 16].

2. *Main Memory:* Implemented using *DRAM* (Dynamic Random Access Memory) [13, 14].

- * *DRAM Characteristics:* Cheaper and denser than SRAM, but slower [14, 16]. Stores charge in a capacitor and must be *periodically refreshed* (hence "dynamic") [16, 17].

3. *Secondary Storage (Slowest/Largest):*

- * *Flash Memory:* Nonvolatile memory used in Personal Mobile Devices [13, 14]. Writes can wear out the bits, necessitating controllers that use *wear leveling* (spreading writes across blocks) [18].

- * *Magnetic Disk:* Used for the largest and slowest level in servers [13, 14]. Mechanical device, thus much slower than semiconductor memory (100,000 times slower than DRAM) but much cheaper per bit [19, 20].

4. *Disk Access Components:* Accessing data on a magnetic disk is a three-stage process involving mechanical delay [21].

- * *Seek Time:* Time to position the read/write head over the correct *track* [21].

- * *Rotational Latency:* Time for the desired *sector* to rotate under the head (average is half the rotation time) [22, 23].

- * *Transfer Time:* Time to transfer the block of bits (function of sector size, rotation speed, and recording density) [24].

IV. Cache Organization and Addressing

1. *Address Mapping Types (Placement):* Dictates where a block from memory can be placed in the cache [25, 26].

- * *Direct-Mapped Cache:* Each memory block maps to *exactly one* cache location [27-29]. Indexing is determined by (Block address) modulo (Number of blocks in the cache) [29, 30]. Simplest and fastest lookup (only one comparator needed) [31].

- * *Set-Associative Cache:* A block maps to a limited number of locations (N) within a specific *set* [32]. The set is found by indexing, and all blocks within that set are searched in parallel [32, 33]. Provides a middle ground between speed and flexibility [32].

* **Fully Associative Cache:** A block can be placed in *any* location in the cache (effectively one set) [34, 35]. Requires searching all cache tags in parallel, making it expensive and practical only for very small caches [34].

2. **Address Fields:** For direct-mapped or set-associative caches, the address is broken down [36]:

* **Tag:** The upper address bits used to identify which memory block currently resides in the cache location [30, 37].

* **Index:** Used to select the specific set or block in the cache [36, 38].

* **Block Offset:** Used to select the desired data within the block [36].

3. **Valid Bit:** A field indicating whether the associated block contains valid data [39].

V. Cache Operation: Miss Handling and Writes

1. **Handling Instruction Misses (Four Steps):**

1. Send the Program Counter (PC) value (minus 4) to memory [40, 41].

2. Instruct memory to read and wait for the response [41].

3. Write the cache entry (data, tag, and set the valid bit ON) [41].

4. Restart instruction execution, which now finds the instruction in the cache (hit) [41].

2. **Write Policies (Handling Data Updates):**

* **Write-Through:** Updates are written simultaneously to the cache and the next lower level of the hierarchy (e.g., main memory) [42-44]. This maintains consistency but can be slow [42, 45].

* **Write Buffer:** Essential for write-through caches; it stores data temporarily while waiting to be written to memory, allowing the processor to continue execution [43].

* **Write-Back:** Updates are written only to the cache block [44, 46]. The modified block is written back to the lower level *only when it is replaced* [46]. Improves performance by reducing memory traffic but is more complex to implement [46-48].

3. **Replacement Policies (for Associative Caches):** Determine which block to evict on a miss [49, 50].

* **Least Recently Used (LRU):** Replaces the block that has been unused for the longest time [51-53]. This is the most effective policy but is costly and often approximated in hardware for higher associativity caches [53].

* **Random:** Blocks are chosen randomly for replacement [54]. Simple to implement and generally competitive with simple LRU approximations [55].

VI. Performance Analysis and Improvement Techniques

1. **Average Memory Access Time (AMAT):** A metric used to evaluate cache designs, combining hit time and miss effects [56].

$$\text{AMAT} = \text{Time for a hit} + (\text{Miss rate} \times \text{Miss penalty})$$

[56]

2. **Impact of Design Choices:**

- Larger Block Size:** Generally decreases the miss rate by exploiting spatial locality, but it increases the miss penalty (transfer time). If the penalty increases too much, overall performance decreases [57-59].

- Increased Associativity:** Decreases the miss rate by reducing conflict misses, but it may increase the hit time due to the complexity of parallel searching and selecting [33, 60, 61].

3. **Multilevel Caches (L1, L2, etc.):** Used to bridge the increasing gap between processor speed and DRAM access time by providing a faster penalty path for L1 misses [62, 63].

- L1 Cache Focus:** Concerned primarily with minimizing **hit time** to support the fast processor clock cycle [64, 65].

- L2 Cache Focus:** Concerned primarily with minimizing the **miss rate** to main memory [64, 65]. It is typically much larger than L1, often having higher associativity and larger block sizes [66].

4. **Software Optimization (Blocking):** A technique, especially for array processing like matrix multiplication, where algorithms operate on small **submatrices or blocks** [67, 68]. The goal is to improve **temporal locality** by ensuring that data loaded into the cache is reused extensively before being replaced, dramatically reducing capacity misses [67, 69].

VII. The Three Cs Model (Sources of Misses)

All cache misses can be qualitatively categorized into three groups, useful for analyzing design improvements [70, 71]:

1. **Compulsory Misses (Cold-start misses):** Misses caused by the **first access** to a block [71, 72].

- Reduction Strategy:** Increase **block size** [61, 73].

2. **Capacity Misses:** Misses that occur because the cache, even if fully associative, **cannot contain all the blocks needed** during execution [71, 72].

- Reduction Strategy:** **Enlarge the cache size** [61, 73].

3. **Conflict Misses (Collision misses):** Misses that occur in direct-mapped or set-associative caches because **multiple blocks map to the same set** and compete for the same location [71, 72]. These are eliminated in a fully associative cache of the same size [72, 74].

- Reduction Strategy:** Increase **associativity** [50, 61].